

Marwadi University

BCA Semester – 3

**05BC3301 – Database Management System – 2
(DBMS-2)**

Unit – 3: Error Handling



Unit – 3: Error Handling

3

Exception Handling:

Exception Types, Predefined Exception, User Defined Exception, Use of `Raise_Application_Error`.

Topics

1. What is Exception ?
2. Syntax of Exception Handling in PL/SQL
3. Types of Exception
 1. System defined
 - 1.1 Named system exceptions
 - Examples of Named system exceptions
 - 1.2 Unnamed system exceptions
 - Examples of Unnamed system exceptions
 2. User Defined
 - Examples of User defined exceptions
4. Exception Trapping Functions
5. RAISE_APPLICATION_ERROR () Procedure

What is Exception ?

- ***Exceptions*** are abnormal condition that can be occur during the execution of program.
- In simple words an ***exception is an error.***
- ***Exceptions are error situations.*** The error situations can be user defined or system exceptions, which are automatically raised by PL/SQL.
- Exceptions are used to handled ***run-time errors*** in PL/SQL. And to handled that exception PLSQL specified exception block is called ***Exception Block.***

Syntax of Exception Handling in PL/SQL

DECLARE

<declarations section>

BEGIN

<executable command(s)>

EXCEPTION

```
<exception handling goes here >
```

WHEN exception1 THEN

exception1-handling-statements

WHEN exception2 THEN

exception2-handling-statements

WHEN exception3 THEN

exception3-handling-statements

• • • • •

WHEN others THEN

exception4-handling-statements

END;

Declaration

Begin

Exception Block

***Write Here
Exception Code***

End;

Types of Exceptions

- There are basically two types of exceptions are there:

- 1. System Defined**

- 2. User Defined**

Types of Exceptions (System Defined)

1. System defined exceptions:

- Defined and maintained *implicitly* by the Oracle, these exceptions are mainly defined in the Oracle Standard Package.
- Whenever an exception occurs inside the program, Oracle *matches and identifies the appropriate exception* from the available set of exceptions.
- Basically, these exceptions are predefined in PL/SQL which gets raised **WHEN** particular database rule is violated.

Types of Exceptions (System Defined)

1. System defined exceptions:

The **System-defined exceptions** are further divided into two categories:

1.1. Named system exceptions

1.2. Unnamed system exceptions

Types of Exceptions (System Defined)

1.1. Named system exceptions

- The named PL/SQL exceptions are ***named in the standard package of PL/SQL***, hence the developer does not need to define the PL/SQL exceptions in their code.
- PL/SQL provides many ***pre-defined named exceptions***, which are executed when any database rule is violated by a program. The following are some of the important pre-defined exceptions.

Types of Exceptions (System Defined)

Exception	Oracle Error	Description
DUP_VAL_ON_INDEX	00001	It is raised when duplicate values are attempted to be stored in a column with a unique index.
INVALID_CURSOR	01001	It is raised when attempts are made to make a cursor operation that is not allowed, such as closing an unopened cursor.
CURSOR_ALREADY_OPEN	06511	This exception is raised when user tries to open a cursor that is already open
INVALID_NUMBER	01722	It is raised when the conversion of a character string into a number fails because the string does not represent a valid number.
LOGIN_DENIED	01017	It is raised when a program attempts to log on to the database with an invalid username or password.

Types of Exceptions (System Defined)

NO_DATA_FOUND	01403	It is raised when a SELECT INTO statement returns no rows.
NOT_LOGGED_ON	01012	It is raised when a database call is issued without being connected to the database.
PROGRAM_ERROR	06501	It is raised when PL/SQL has an internal problem.
ROWTYPE_MISMATCH	06504	It is raised when a cursor fetches value in a variable having incompatible data type.

Types of Exceptions (System Defined)

STORAGE_ERROR	06500	It is raised when PL/SQL ran out of memory or memory was corrupted.
TOO_MANY_ROWS	01422	It is raised when a SELECT INTO statement returns more than one row.
VALUE_ERROR	06502	It is raised when an arithmetic, conversion, truncation, or size constraint error occurs.
ZERO_DIVIDE	01476	It is raised when an attempt is made to divide a number by zero.
OTHERS	--	This flag is used to catch any error situation not coded by the programmer in the exception section. Therefore, it must appear last in the exception section.

Types of Exceptions (System Defined)

Example 1:

-- Write a program that explains the use of NO_DATA_FOUND exception.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    SALARY NUMBER;
```

```
BEGIN
```

```
    SELECT SAL INTO SALARY FROM EMP WHERE EMPNO = 100;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE ('No records are found for empno 100');
```

```
END;
```

```
/
```

Types of Exceptions (System Defined)

Example 2:

-- Write a program that explains the use of TOO_MANY_ROWS exception.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    SALARY NUMBER;
```

```
BEGIN
```

```
    SELECT SAL INTO SALARY FROM EMP WHERE DEPTNO = 10;
```

```
EXCEPTION
```

```
    WHEN TOO_MANY_ROWS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('More than one rows are selected for  
department 10');
```

```
END;
```

```
/
```

Types of Exceptions (System Defined)

Example 3:

--Write a program that explains the use of INVALID_NUMBER exception.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    NO NUMBER (5);
```

```
BEGIN
```

```
    SELECT TO_NUMBER ('A1') INTO NO FROM DUAL;
```

```
EXCEPTION
```

```
    WHEN INVALID_NUMBER THEN
```

```
        DBMS_OUTPUT.PUT_LINE ('Wrong Data Type/ Data conversion');
```

```
END;
```

```
/
```

Types of Exceptions (System Defined)

Example 4:

--Write a program that explains the use of ZERO_DIVIDE exception.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    NO NUMBER (3);
```

```
BEGIN
```

```
    SELECT 100/0 INTO NO FROM DUAL;
```

```
EXCEPTION
```

```
    WHEN ZERO_DIVIDE THEN
```

```
        DBMS_OUTPUT.PUT_LINE ('You can not divide the number by 0.');
```

```
END;
```

```
/
```

Types of Exceptions (System Defined)

1.2. Unnamed System Exceptions

- The system exceptions for which *Oracle does not have a name* are known as unnamed system exceptions. *These exceptions do not occur frequently* and are written with a code and an associated message.
- Using the **WHEN OTHERS** exception handler we can handle the unnamed exceptions.

Types of Exceptions (System Defined)

Example 5:

```
SET SERVEROUTPUT ON
```

```
BEGIN
```

```
    DBMS_OUTPUT.put_line(TO_DATE      ('2010      10      10  
44:55:66', 'YYYYSS'));
```

```
EXCEPTION
```

```
    WHEN others THEN
```

```
        DBMS_OUTPUT.put_line('Bad date format');
```

```
END;
```

```
/
```

Types of Exceptions (User Defined)

2. User defined exceptions:

- Apart from system exceptions we can explicitly define ***exceptions based on business rules***. These are known as ***user-defined exceptions***.
- Like all other programming languages, Oracle also allows you to ***declare and implement your own exceptions***.
- Unlike System defined exceptions, ***these exceptions are raised explicitly in the PL/SQL block***.

Types of Exceptions (User Defined)

- Steps to be followed to use user-defined exceptions:

Step - 1:

They should be explicitly ***declared*** in the declaration section.

Step - 2:

They should be explicitly ***raised*** in the Execution Section.

Step - 3:

They should be ***handled*** by referencing the user-defined exception name in the exception section.

Types of Exceptions (User Defined)

Example 6:

-- Write a program to using implicit cursor to display the commission of given EMPNO, also raise a user defined exception NULL _COMMISSION when no value (NULL) is available for commission.

(Use User Defined Exception)

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    VCOMMISSION EMP.COMM%TYPE;
```

```
    NULL_COMMISSION EXCEPTION;
```

```
BEGIN
```

```
    SELECT COMM INTO VCOMMISSION FROM EMP WHERE EMPNO = &EMPNO;  
    DBMS_OUTPUT.PUT_LINE('The commission value is : - ' || VCOMMISSION);
```

```
    IF VCOMMISSION IS NULL THEN
```

```
        RAISE NULL_COMMISSION;
```

```
    END IF;
```

```
EXCEPTION
```

```
    WHEN NULL_COMMISSION THEN
```

```
        DBMS_OUTPUT.PUT_LINE ('NO COMMISSION FOUND FOR GIVEN EMPLOYEE');
```

```
END;
```

```
/
```

Types of Exceptions (User Defined)

Example 7:

-- Write a program using a cursor to insert the records of employee in EMP_BACKUP table for given DEPT_NO, also raise a user defined exception NO_DEPT_FOUND when no records are found for entered DEPT_NO. (Use User Defined Exception)

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    I          NUMBER(3);  
    VENO       EMP.EMPNO%TYPE;  
    VENAME     EMP.ENAME%TYPE;  
    VSAL       EMP.SAL%TYPE;  
    VDNO       EMP.DEPTNO%TYPE;
```

```
    CURSOR CUR_EMP IS SELECT EMPNO, ENAME, SAL FROM EMP WHERE  
    DEPTNO = &DEPTNO;
```

```
    NO_DEPT_FOUND EXCEPTION;
```

Types of Exceptions (User Defined)

```
BEGIN
```

```
  I:=0;
```

```
  OPEN CUR_EMP;
```

```
  LOOP
```

```
    FETCH CUR_EMP INTO VENO, VENAME, VSAL;
```

```
    EXIT WHEN CUR_EMP%NOTFOUND;
```

```
    I:=I+1;
```

```
    DBMS_OUTPUT.PUT_LINE (VENO || ' ' || VENAME || ' ' || VSAL);
```

```
    INSERT INTO EMP_BACKUP (EMPNO,ENAME,SAL) VALUES  
(VENO,VENAME,VSAL);
```

```
  END LOOP;
```

Types of Exceptions (User Defined)

```
IF CUR_EMP%ROWCOUNT = 0 THEN
```

```
    CLOSE CUR_EMP;
```

```
    RAISE NO_DEPT_FOUND;
```

```
END IF;
```

```
    DBMS_OUTPUT.PUT_LINE('TOTAL NO. OF RECORDS INSERTED INTO  
EMP_BACKUP TABLE ARE:- ' || I);
```

```
CLOSE CUR_EMP;
```

```
EXCEPTION
```

```
    WHEN NO_DEPT_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE ('NO RECORDS ARE FOUND FOR GIVEN DEPT NO');
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE ('PL/SQL CODE ENCOUNTERED AN ERROR');
```

```
END;
```

```
/
```

Exception Trapping Functions

- When an exception occurs in your program, ***you can find the error code for the error and its associated message.*** Once you know the error code and the message, you can modify your program to take action based on the error. The two functions to identify the error code and error message.
- **SQLCODE:** The SQLCODE function returns the number of the error code. The number can be assigned to a variable of NUMBER datatype.
- **SQLERRM:** The SQLERRM function returns the error message associated with the error code. It can be assigned to a VARCHAR2 type variable.

Exception Trapping Functions

Example 8:

-- Write a program that explains the use of exception trapping functions SQLCODE and SQLERRM.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    VDEPTNO    DEPT.DEPTNO%TYPE;
```

```
    VDNAMN     DEPT.DNAME%TYPE;
```

```
    VLOC       DEPT.LOC%TYPE;
```

```
    V_CODE     VARCHAR2(255); --variable to fetch and hold error code
```

```
    V_MESSAGE  VARCHAR2(255); --variable to get error message
```

Exception Trapping Functions

Example 8:

BEGIN

```
SELECT  DEPTNO,DNAME,LOC INTO  VDEPTNO, VDNAME, VLOC  
FROM DEPT WHERE DEPTNO = &DEPTNO;
```

```
IF SQL%ROWCOUNT > 0 THEN
```

```
    INSERT INTO NEWDEPT VALUES (VDEPTNO, VDNAME, VLOC);
```

```
    DBMS_OUTPUT.PUT_LINE('Records is inserted into NEWDEPT  
table for given deptno ');
```

```
    DBMS_OUTPUT.PUT_LINE (VDEPTNO || ' ' || VDNAME || ' ' ||  
VLOC);
```

```
END IF;
```

Exception Trapping Functions

Example 8:

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        V_CODE:=SQLCODE;
```

```
        V_MESSAGE:=SQLERRM;
```

```
        DBMS_OUTPUT.PUT_LINE ('ERROR CODE: = ' || V_CODE);
```

```
        DBMS_OUTPUT.PUT_LINE ('ERROR MESSAGE: = ' || V_MESSAGE);
```

```
END;
```

```
/
```

Raise_Application_Error () Procedure

- ***RAISE_APPLICATION_ERROR*** is a built-in procedure in Oracle which is used to display the user-defined error messages along with the error number whose *range is in between -20000 and -20999*.
- Whenever a message is displayed using ***RAISE_APPLICATION_ERROR***, all previous transactions ***which are not committed within the PL/SQL Block are rolled back automatically*** (i.e. change due to INSERT, UPDATE, or DELETE statements).
- ***RAISE_APPLICATION_ERROR*** raises an exception but does not handle it.
- ***RAISE_APPLICATION_ERROR*** is used for the following reasons,
 - a) to create a unique id for a user-defined exception.
 - b) to make the user-defined exception look like an Oracle error.

Raise_Application_Error () Procedure

- The General Syntax to use this procedure is:

RAISE_APPLICATION_ERROR (error_number, error_message);

- The error_number must be between -20000 and -20999.
- The error_message is the message you want to display when the error occurs.

- Steps to be followed to use RAISE_APPLICATION_ERROR procedure:

1. Declare a user-defined exception in the declaration section.
2. Raise the user-defined exception based on a specific business rule in the execution section.
3. Finally, catch the exception and link the exception to a user-defined error number in RAISE_APPLICATION_ERROR.

Raise_Application_Error () Procedure

Example 9:

--Write a program that explains the use of ZERO_DIVIDE exception without exception block.

SET SERVEROUTPUT ON

DECLARE

 N NUMBER (3);

 VD NUMBER(3);

BEGIN

 VD:=&VD;

 IF VD=0 THEN

 RAISE_APPLICATION_ERROR(-20230,'You can not divide any number with zero');

 ELSE

 SELECT 100/VD INTO N FROM DUAL;

 DBMS_OUTPUT.PUT_LINE(N);

 END IF;

END;

/

Raise_Application_Error () Procedure

Example 10:

--Write a program that explains the use of ZERO_DIVIDE exception using exception block.

SET SERVEROUTPUT ON

DECLARE

 N NUMBER (3);

 VD NUMBER(3);

BEGIN

 VD:=&VD;

 SELECT 100/VD INTO N FROM DUAL;

 DBMS_OUTPUT.PUT_LINE(N);

EXCEPTION

 WHEN OTHERS THEN

 RAISE_APPLICATION_ERROR(-20230,'You can not divide any number with zero');

END;

/

Raise_Application_Error () Procedure

Example 11:

--Write a program that explains the use of ZERO_DIVIDE exception.

SET SERVEROUTPUT ON

DECLARE

N NUMBER (3);

N1 NUMBER (3);

VD NUMBER(3);

E1 EXCEPTION;

E2 EXCEPTION;

BEGIN

VD:=&VD;

N1:=&N1;

IF VD=0 THEN

 RAISE E1;

Raise_Application_Error () Procedure

Example 11:

--Write a program that explains the use of ZERO_DIVIDE exception. (Continue...)

```
ELSIF VD=5 THEN
```

```
    RAISE E2;
```

```
ELSE
```

```
    SELECT N1/VD INTO N FROM DUAL;
```

```
    DBMS_OUTPUT.PUT_LINE(N);
```

```
END IF;
```

```
EXCEPTION
```

```
    WHEN E1 THEN
```

```
        RAISE_APPLICATION_ERROR(-20230,'Incorrrect Divisor.');
```

```
    WHEN E2 THEN
```

```
        RAISE_APPLICATION_ERROR(-20231,'Incorrect Divisor-5.');
```

```
END;
```

```
/
```

PRAGMA EXCEPTION_INIT

- PL/SQL user named exception.
- you can define your own error message and error number using Pragma EXCEPTION_INIT or RAISE_APPLICATION_ERROR function.
- Pragma is a keyword directive to execute proceed at compile time.

PRAGMA EXCEPTION_INIT

- pragma EXCEPTION_INIT function take this two arguments:
 - exception_name
 - error_number
- define pragma EXCEPTION_INIT in DECLARE BLOCK on your program.
- Syntax:

```
PRAGMA EXCEPTION_INIT(exception_name, -error_number);
```

PRAGMA EXCEPTION_INIT

Example 12:

Write a program that explains the use of ZERO_DIVIDE exception using PRAGMA EXCEPTION_INIT.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    NO1 NUMBER:=&NO1;
```

```
    NO2 NUMBER:=&NO2;
```

```
    RESULT NUMBER;
```

```
    EX1 EXCEPTION;
```

```
    PRAGMA EXCEPTION_INIT(EX1,-100);
```

```
BEGIN
```

```
    IF NO2=0 THEN
```

```
        RAISE EX1;
```

```
    ELSE
```

```
        RESULT:=NO1/NO2;
```

```
        DBMS_OUTPUT.PUT_LINE('RESULT IS = ' || RESULT);
```

```
    END IF;
```

PRAGMA EXCEPTION_INIT

Example 12:

Write a program that explains the use of ZERO_DIVIDE exception using PRAGMA EXCEPTION_INIT. (Continue...)

```
EXCEPTION
    WHEN EX1 THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: DIVIDE WITH ZERO.');
```

END;
/